Московский авиационный институт
(национальный исследовательский университет)

Факультет компьютерных наук и прикладной математики

Кафедра вычислительной математики и программирования

Курсовая работа по курсу «Дискретный анализ»

Сжатие данных методом LZW

Студент: Волков А.А.
Преподаватель:
Группа: М8О-301БВ-24
Дата: 11 ноября 2026 г.
Оценка:
Подпись:

Москва, 2026

Курсовая работа. Сжатие данных методом LZW

Задача: Разработать кодирование и декодирование строк методом LZW. Алфавит состоит из строчных латинских букв. Начальные коды: a — 0, b — 1, . . . , z — 25. Код EOF равен 26, новые записи начинаются с 27. В режиме compress вводится строка, выводятся числовые коды. В режиме decompress вводятся коды до EOF, выводится исходная строка. Например, abracadabra соответствует последовательности 0 1 17 0 2 0 3 27 29 26.

1 Описание реализации

Кодировщик хранит словарь соответствий строк и кодов в `std::unordered_map`. Рабочая строка расширяется очередным символом. Если расширение отсутствует в словаре, выводится код прежней строки, расширение добавляется под следующим свободным кодом, а рабочей строкой становится прочитанный символ. После обработки выводятся последняя строка и EOF.

Декодировщик восстанавливает словарь самостоятельно. Первый код задаёт начальную строку. Для каждого следующего кода определяется текущая строка, а новая запись состоит из предыдущей строки и первого символа текущей. Если получен ещё не внесённый следующий код, текущая строка равна предыдущей с добавленным её первым символом. Пример такого случая: aaa, коды 0 27 26.

При условии постоянного времени операций хеш-таблицы декодирование выполняется за $O(N)$, где N — длина восстановленной строки. Добавление символов к результату учитывается амортизированно.

2 Тест производительности

Для сравнения выбран DEFLATE из библиотеки zlib: он сочетает поиск повторов LZ77 и коды Хаффмана и служит готовым библиотечным ориентиром [3, 4]. Второй метод, RLE, — распространённое кодирование серий одинаковых символов парой «символ, количество повторений» [2]. Он позволяет сравнить словарное сжатие с подходом, эффективным на длинных сериях.

Команда сборки:

```
g++ -std=c++20 -O2 benchmark.cpp -lz -o benchmark
```

Запуск: `./benchmark 5 > results.csv`.

Для каждой пары размера и типа данных выполнен прогрев и пять повторений. Приведена медиана отдельно для сжатия и восстановления. Создание словарей, выделение выходных буферов, сериализация и восстановление входят в замеры.

В benchmark.cpp используется копия реализации LZW из main.cpp с возвратом результата вместо консольного вывода; перевод массива кодов в байты и обратно также входит в измеряемое время. Генерация, консольный вывод и проверка равенства исходной и восстановленной строк не входят.

Использованы следующие типы данных:

- `constant` — строка из одинаковых букв a;
- `periodic` — повторение abracadabra с обрезкой до нужной длины;
- `random` — независимый равномерный выбор из 26 строчных латинских букв.

Для случайных строк используется генератор псевдослучайных чисел `mt19937` с одинаковым начальном сидом. Для каждого размера генератор инициализируется заново.

Форматы. LZW хранит каждый код, включая EOF, четырьмя байтами. RLE хранит символ одним байтом и длину серии четырьмя байтами. DEFLATE измеряется в оболочке zlib, включая её заголовок и контрольную сумму. Сжатие выполняется `compress2` с заданным уровнем, восстановление — `uncompress` [4]. Исходный размер передаётся декодерам отдельно; размер контейнера с этим метаданным не учитывается ни для одного метода. Сравнивается размер полезного двоичного представления, а не текстовых чисел.

Отношение $R = S_{\text{encoded}}/S_{\text{input}}$ меньше единицы означает сжатие. В таблице времена указаны в миллисекундах, размеры — в байтах. Порядок методов фиксирован.

Тип	Вход	Метод	Выход	R	Сжатие	Восст.
constant	1000	DEFLATE	17	0.017000	0.037	0.004
constant	1000	LZW	184	0.184000	0.028	0.007
constant	1000	RLE	5	0.005000	0.002	0.001

Тип	Вход	Метод	Выход	R	Сжатие	Восст.
periodic	1000	DEFLATE	26	0.026000	0.038	0.002
periodic	1000	LZW	552	0.552000	0.028	0.011
periodic	1000	RLE	4550	4.550000	0.006	0.005
random	1000	DEFLATE	629	0.629000	0.044	0.005
random	1000	LZW	2884	2.884000	0.057	0.033
random	1000	RLE	4790	4.790000	0.006	0.005
constant	10000	DEFLATE	34	0.003400	0.052	0.018
constant	10000	LZW	568	0.056800	0.334	0.015
constant	10000	RLE	5	0.000500	0.005	0.001
periodic	10000	DEFLATE	51	0.005100	0.058	0.007
periodic	10000	LZW	1836	0.183600	0.305	0.044
periodic	10000	RLE	45455	4.545500	0.038	0.031
random	10000	DEFLATE	6255	0.625500	0.132	0.026
random	10000	LZW	20340	2.034000	0.579	0.293
random	10000	RLE	48065	4.806500	0.042	0.033
constant	100000	DEFLATE	121	0.001210	0.290	0.171
constant	100000	LZW	1792	0.017920	5.080	0.062
constant	100000	RLE	5	0.000050	0.038	0.002
periodic	100000	DEFLATE	226	0.002260	0.275	0.053
periodic	100000	LZW	5892	0.058920	3.621	0.139
periodic	100000	RLE	454550	4.545500	0.511	0.299
random	100000	DEFLATE	63454	0.634540	1.643	0.340
random	100000	LZW	154944	1.549440	6.027	2.479
random	100000	RLE	480775	4.807750	0.406	0.337
constant	1000000	DEFLATE	992	0.000992	2.473	1.644
constant	1000000	LZW	5660	0.005660	97.775	0.393
constant	1000000	RLE	5	0.000005	0.309	0.014
periodic	1000000	DEFLATE	1969	0.001969	3.244	0.520
periodic	1000000	LZW	18720	0.018720	58.515	0.503
periodic	1000000	RLE	4545455	4.545455	3.440	2.931
random	1000000	DEFLATE	635630	0.635630	17.818	3.245
random	1000000	LZW	1253616	1.253616	96.875	31.262
random	1000000	RLE	4807365	4.807365	4.194	3.277

3 Выводы

Под наилучшим результатом по степени сжатия далее понимается минимальный размер закодированного представления. Скорость рассматривается отдельно.

1. На наборе одинаковых символов наилучшим оказался RLE: вся строка представляется одной серией. Он также показал наименьшее время сжатия и восстановления на всех проверенных размерах.

2. На случайных строках наилучший размер показал DEFLATE. Алфавит ограничен строчными латинскими буквами, поэтому даже при независимом выборе символов их восьмибитное хранение содержит избыточность, которую невелируют коды Хаффмана. RLE быстрее сжимает эти данные, но увеличивает их размер; LZW также увеличивает размер.

3. На периодических строках наилучший размер показал DEFLATE: повторы блоков подходят для ссылок LZ77. LZW также сокращает размер, а RLE не использует повторение целых блоков и расширяет данные.

Применимость LZW

LZW целесообразно рассматривать для данных с повторяющимися подстроками, включая структурированные текстовые записи и повторяющиеся блоки. Преимущество особенно заметно, если требуется уменьшить размер периодических данных по сравнению с простым кодированием серий.

Для сравнения размеров используется процент $100(S_{\mathrm{LZW}} - S_{\mathrm{base}})/S_{\mathrm{base}}$: отрицательное значение означает уменьшение относительно метода сравнения. Все следующие диапазоны получены из попарных сравнений на одинаковых входах для четырёх проверенных размеров.

На периодических данных представление LZW меньше RLE на 87.87–99.59%, но больше DEFLATE на 850.74–3500.00%. В данной реализации выигрыш в размере относительно RLE оплачивается более длительным кодированием.

На случайных строках LZW занимает на 39.79–73.92% меньше места, чем RLE, но на 97.22–358.51% больше, чем DEFLATE. При этом LZW всё равно расширяет исходные данные. Для длинных серий одного символа предпочтительнее RLE; LZW здесь не даёт выигрыша в размере или скорости.

При увеличении длины периодического набора от 1000 до 1000000 символов доля размера LZW относительно исходного уменьшается с 55.20% до 1.872%. Для случайного набора той же длины доля уменьшается с 288.40% до 125.3616%, но остаётся выше исходного размера. Это показывает пользу накопленного словаря и одновременно ограниченность результата на случайных данных.

Хранение полных строк в словаре также увеличивает стоимость копирования и хеширования. Поэтому для практического сжатия исследованных периодических и случайных данных по полученным результатам предпочтительнее zlib; для длинных однородных серий — RLE. LZW остаётся обоснованным словарным подходом, однако преимущество иной, оптимизированной реализации потребовало бы отдельного эксперимента.

Список литературы

[1] Викиконспекты ИТМО. Алгоритм LZW. Статья об алгоритме LZW.

[2] Викиконспекты ИТМО. Алгоритм RLE. Статья об алгоритме RLE.

[3] Deutsch P. DEFLATE Compressed Data Format Specification version 1.3. RFC 1951, 1996. `https://www.rfc-editor.org/rfc/rfc1951`.

[4] Gailly J.-l., Adler M. zlib Manual. `https://zlib.net/manual.html`.